Docker Adventure

Milestone 2

Matt Krueger

May 29, 2025

The artifact, a web application developed in CS340, Client Server Development, connects to a local MongoDB instance to display rescue dog data using the model-view-controller design pattern. It comprises three layers: a database layer, an API layer, and a front-end layer.

I chose this item after the suggestion to dockerize the application for the first enhancement. I immediately recognized its potential as a valuable addition to the project due to its diverse services and the dependency on a local database. This enhancement addresses a significant issue with the project: it was developed on a school lab and doesn't function properly on my personal machine without the local database set up for me there.

By successfully dockerizing the project, this enhancement enhances the complexity and usefulness of the development environment. This aligns with my goals of demonstrating technical leadership and product design by improving the development and deployment tooling. It facilitates collaborative development by making local installations easier and standardizing them with a hypothetical production environment. This enhancement successfully meets the outcomes for both 1 and 4.

I encountered some difficulties while completing the enhancement. Initially, I overthought how to execute initialization scripts on the database setup. I assumed I would need to connect to the database and configure it using a client API or command-line tools. However, after some research, I discovered that MongoDB itself has a built-in mechanism for this. All I had to do was add a JavaScript file called init.js to a specific directory. This solution simplified the process significantly.

The next challenge was getting my Python imports to work within the Docker container. This required me to study the Dockerfile and learn how to execute commands during the build step. Understanding Docker Compose made integrating the two Docker containers easy once I grasped the network and volumes.

One concern I had was that when I edited a file in the Jupyter Notebook inside the Docker container through the exposed port, the file edits were out of sync with my project files until I added a line in the volumes specification for that container to link them.

Additionally, I learned a best practice by making the connection string for my MongoDB service an environment variable within the containers that connect to it.